

last command status with the shell variable `$?`. You can use environment variables also to set various options.

Just to give you a taste of this, the example given below is a goss config to verify a GNU/Linux based test server that has at least working ssh login functionality, root formatted with a particular filesystem and not filled up beyond 90%, and Docker engine components properly set up and running. The goss test run verifies our needs versus reality in a report form to catch the misconfiguration early.

```
--- example goss.yaml ---
```

```
kernel-param:
  kernel.ostype:
    value: Linux

mount:
  /:
    exists: true
    filesystem: overlay
    usage:
      lt: 90

port:
  tcp:22:
    listening: true
    ip:
      - 0.0.0.0
  tcp6:22:
    listening: true
    ip:
      - ':::'

user:
  sshd:
    exists: true

package:
  docker-ce:
    installed: true

service:kernel-param:
  kernel.ostype:
    value: Linux

mount:
  /:
    exists: true
    filesystem: overlay
    usage:
      lt: 90
```

```
port:
  sshd:
    enabled: true
    running: true
  docker:
    enabled: true
    running: true

process:
  sshd:
    running: true
  containerd:
    running: true

dns:
  localhost:
    resolvable: true
  addrs:
    consist-of: ["127.0.0.1", ":::1"]
    timeout: 500 # in milliseconds
```

Goss provides more resources than we saw in our basic example above, to build tests covering almost every aspect of a modern running machine. You can browse through the link provided in the References section at the end of the article to know about all the tests currently provided by it. You don't always need to create the goss test configuration from scratch. You can use the *add* command to append a test for a resource. If the mentioned resources are absent on your machine, the added tests ensure they do not exist on the system. It's recommended you use the *add* command on a fully configured system matching your desired end state. Otherwise, you could tweak the resource properties in the generated test configuration as per your need. There is also an *autoadd* command to automatically add many, but not all, existing resources in your server matching the provided argument. Do go through the goss documentation and try these commands.

The dgoss is a shell wrapper over goss to use its capabilities, with a few extra functionalities to create acceptance as code gate in your container image build and release pipelines. The usual way to run dgoss is to download it (and goss, if not already in your test system) and then launch it with your goss test configurations. But we'll use it through a Docker driver image to run it containerised without downloading anything.

First, let's create a dgoss driver image with all the necessary artifacts in it. Execute the command *docker build --no-cache --progress plain . -f Dockerfile_DGossDriver -t dgoss:0.4.9* in your terminal in a working directory, where the dockerfile shown below and *goss.yaml* are located to create the dgoss driver image: